

University Portal

System Architecture

&

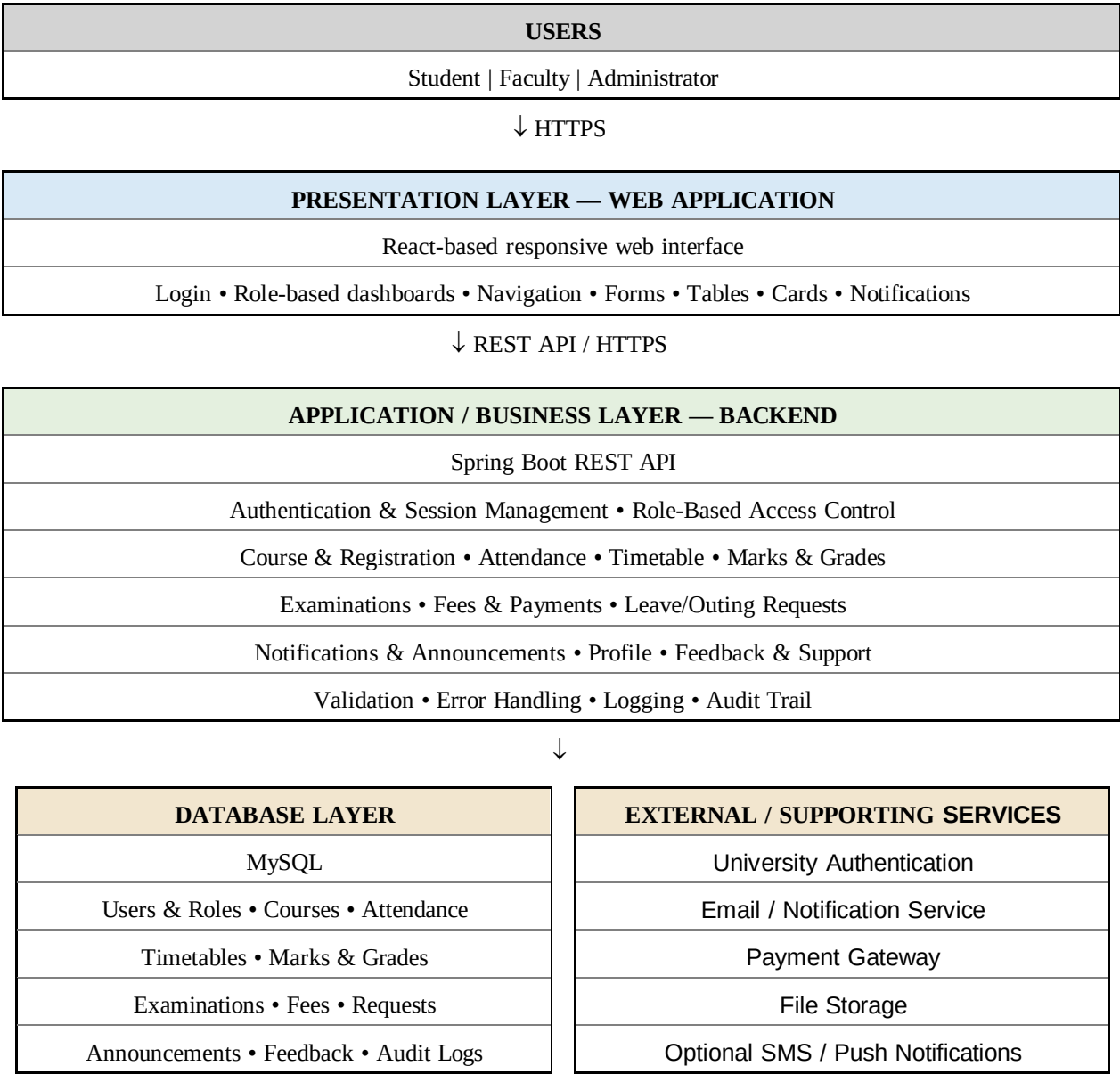
UML Diagrams

Contents:

1. System Architecture
2. Usecase Diagram
3. Class Diagram
4. Sequence Diagram
5. Activity Diagram
6. Communication/Collaboration Diagram

University Portal – System Architecture

- The System Architecture Diagram illustrates the overall structure of the University Portal. It consists of the **Presentation Layer**, **Application/Business Layer**, and **Database Layer**, along with external services. Students, faculty, and administrators access the portal through the web application, while the backend handles authentication, role-based access, academic services, notifications, payments, and communication with the database and external services.



1. Architecture Overview

The University Portal follows a three-tier web application architecture consisting of the Presentation Layer, Application/Business Layer, and Database Layer. External services are connected through secure APIs. This structure separates the user interface from business logic and data storage, making the system easier to maintain, secure, scale and extend.

2. Presentation Layer

The frontend provides a responsive web interface for students, faculty and administrators. React can be used to build reusable components such as dashboards, navigation bars, forms, tables, cards, alerts and notification panels. The interface communicates with the backend through HTTPS-based REST APIs.

3. Application / Business Layer

The backend acts as the central processing layer. Spring Boot can be used to implement REST APIs and business logic. It handles

authentication, authorization, role-based access, validation, request processing, notifications, logging and communication with the database and external services.

4. Core Modules

Authentication & Authorization: Login, session management and role-based permissions for students, faculty and administrators.

Student Module: Courses, registration, attendance, timetable, marks, examinations, fees, leave/outing, notifications, profile and support.

Faculty Module: Assigned courses, timetable, attendance management, permitted student information, announcements and profile.

Administrator Module: User and role management, academic information, announcements, workflows, support, configuration and audit access.

Notification Module: Portal notifications and email-based alerts for important academic and administrative events.

Payment Module: Fee information, payment status and integration with an approved payment gateway.

5. Database Layer

MySQL can be used as the primary relational database. It stores structured institutional data such as users, roles, courses, registrations, attendance, timetables, marks, examinations, fees, requests, announcements, feedback and audit records. Database access should be handled through the backend rather than directly from the frontend.

6. External Services

The system can integrate with a university authentication service, email/notification provider, payment gateway and file storage service. These integrations should be isolated behind backend service modules so that external providers can be replaced without major changes to the frontend.

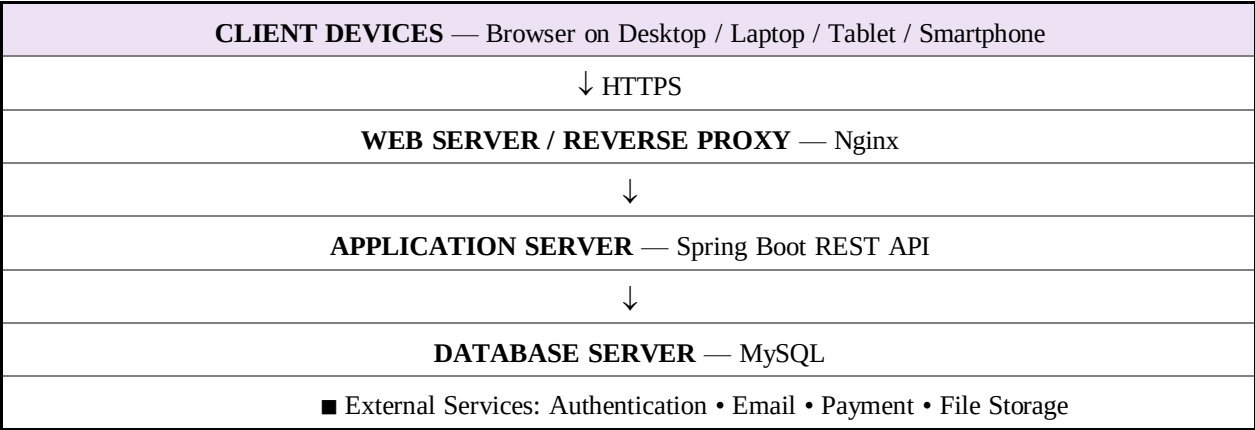
7. Technical Requirements

Component	Technology / Requirement
Frontend	React.js, HTML5, CSS3, JavaScript
Backend	Java Spring Boot, REST APIs
Database	MySQL
Authentication	JWT-based authentication with role-based access control
API Communication	HTTPS + REST/JSON
UI Design	Responsive design for desktop, tablet and mobile
Notifications	Email service and in-portal notifications
Payments	Secure payment gateway integration
File Storage	Cloud/object storage for documents where required
Version Control	Git and GitHub
Deployment	Docker + Linux-based server or cloud deployment
Security	HTTPS, password hashing, input validation, authorization, secure sessions and audit logging

8. Security Architecture

- All client-server communication uses HTTPS.
- JWT-based authentication identifies authenticated users, while role-based access control restricts protected operations.
- Passwords are stored using secure one-way hashing rather than plain text.
- Backend APIs validate input and enforce authorization before accessing protected data.
- Important authentication, administrative and record-changing actions are written to audit logs.
- Sensitive information is protected through database access controls, least-privilege permissions and secure configuration.

9. Deployment Architecture

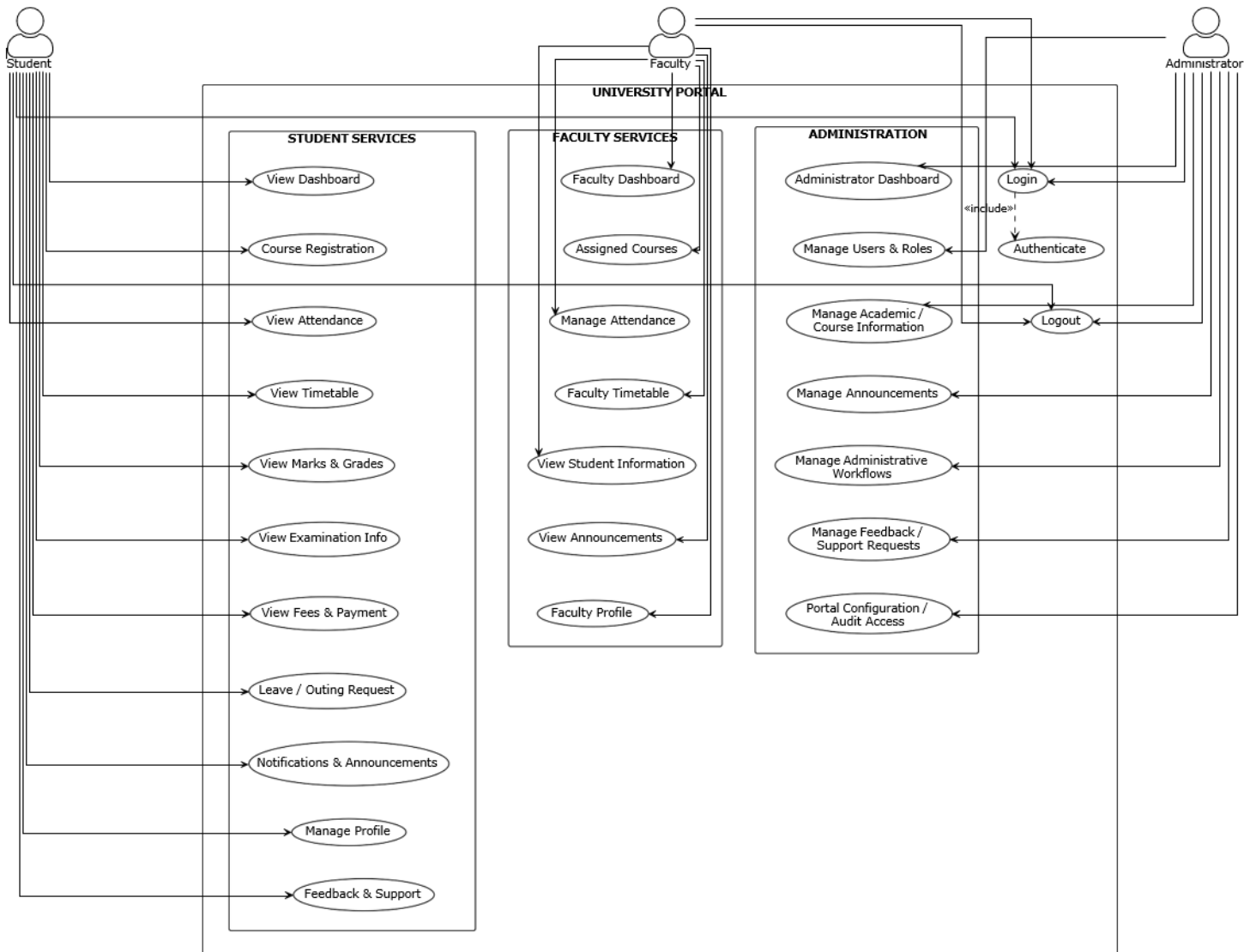


10. Key Benefits

- Modular separation of frontend, backend and data layers
- Centralized role-based security
- Reusable UI and backend components
- Easier maintenance and testing
- Scalable architecture for increasing users and academic records
- Straightforward integration with university and third-party services
- Support for future mobile applications and additional portal module

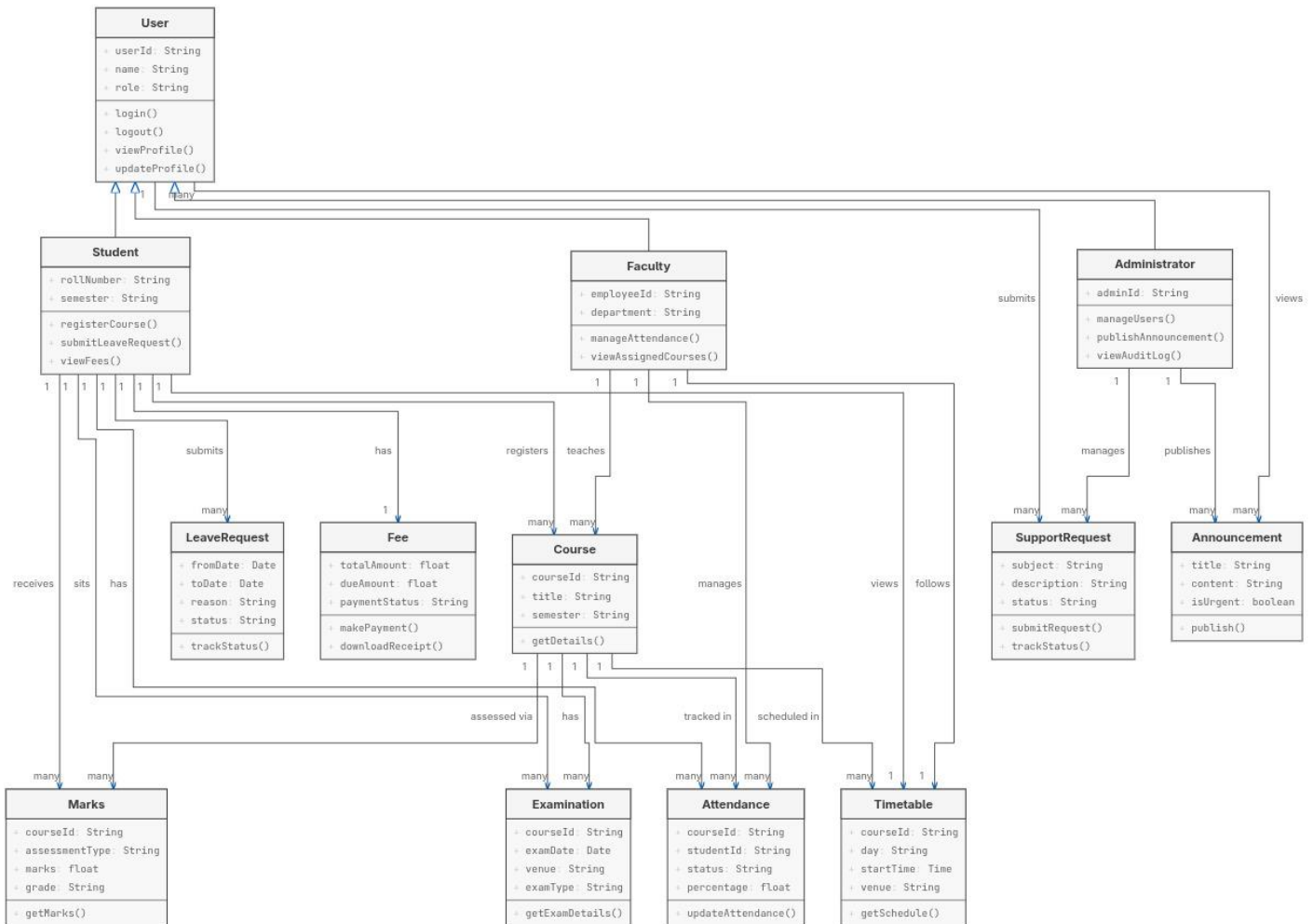
Usecase Diagram

The Use Case Diagram represents the interactions between the three primary users—Student, Faculty, and Administrator—and the University Portal. Students can access courses, attendance, timetable, marks, examinations, fees, leave requests, notifications, profile, and support services. Faculty can manage assigned courses and attendance and access student information, while administrators manage users, academic information, announcements, workflows, support, and portal configuration.



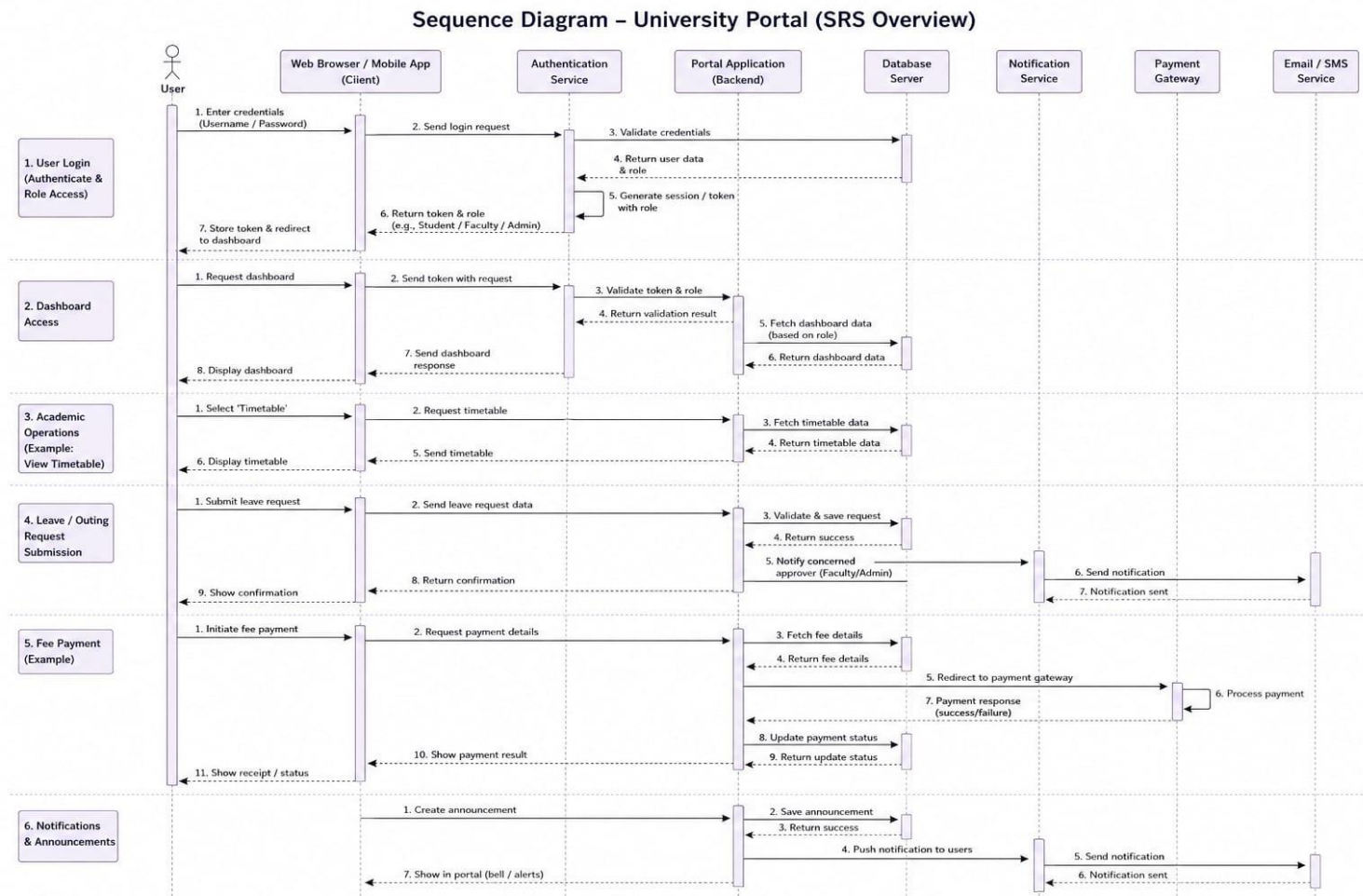
Class Diagram

The Class Diagram represents the major entities and their relationships within the University Portal. It includes classes such as User, Student, Faculty, Administrator, Course, Attendance, Marks, Examination, Timetable, Fee, LeaveRequest, Announcement, and SupportRequest. The diagram shows their attributes, operations, inheritance relationships, and associations required to support the portal's academic and administrative functions.



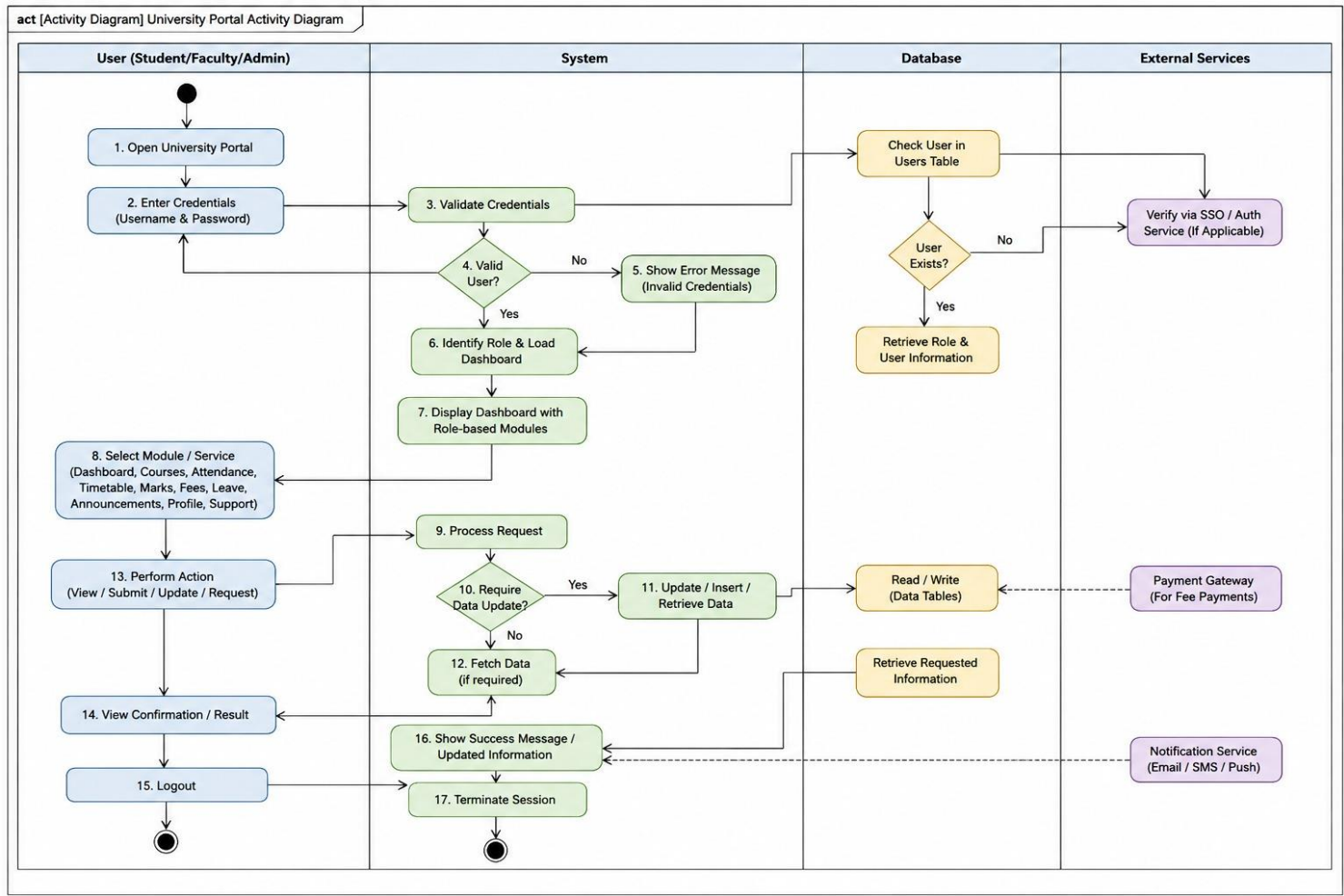
Sequence Diagram

The Sequence Diagram illustrates the chronological flow of interactions between the user, client application, authentication service, portal backend, database server, notification service, payment gateway, and email/SMS service. It demonstrates major operations such as user authentication, dashboard access, academic information retrieval, leave submission, fee payment, and notifications, showing the order in which messages are exchanged.



Activity Diagram

The Activity Diagram represents the overall workflow of the University Portal. The process begins when a user opens the portal and enters their credentials. The system validates the user, identifies the appropriate role, and loads the corresponding dashboard. The user can then select a service, perform an action, retrieve or update information, view the result, and finally log out of the system.

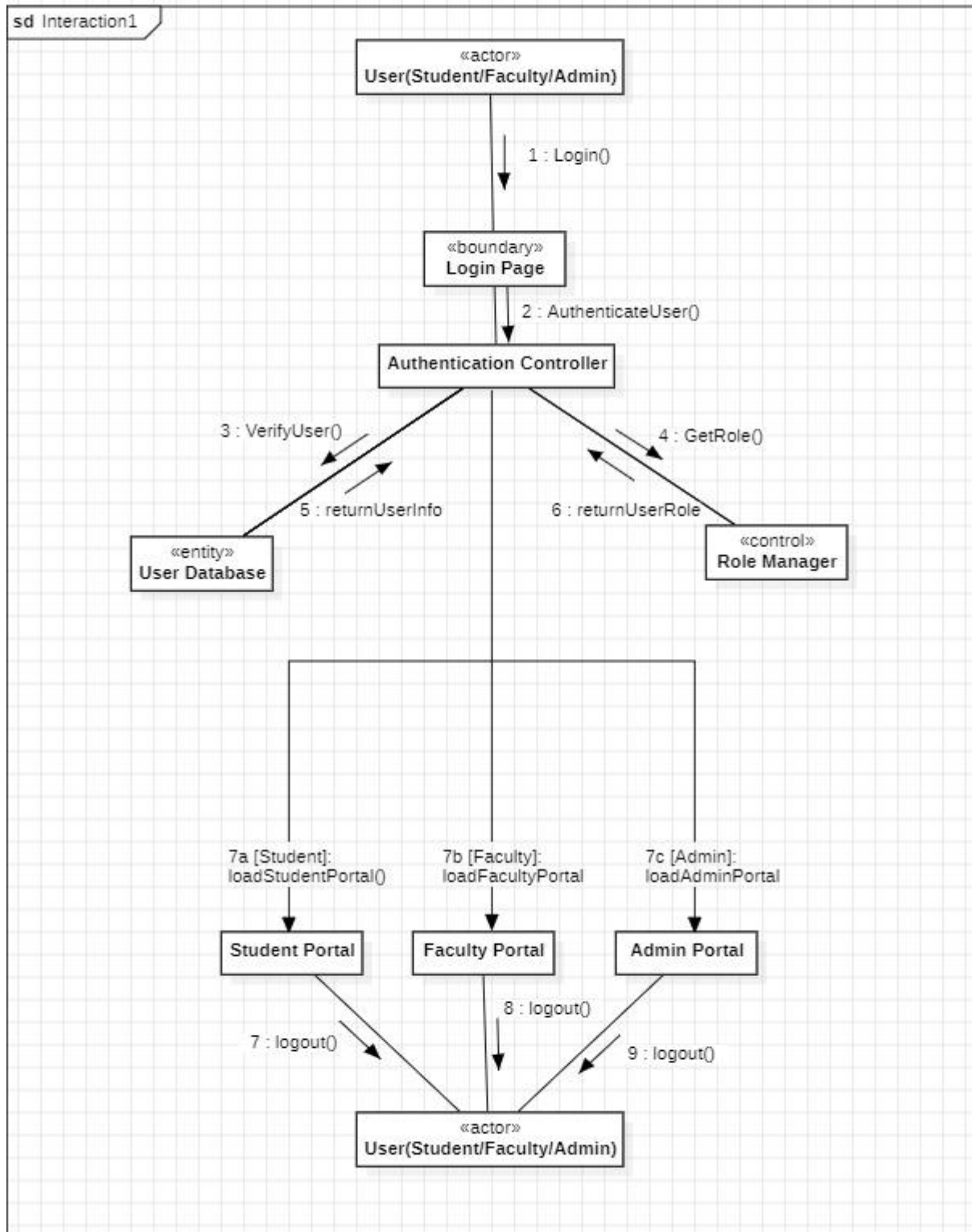


Communication/Collaboration Diagram

The Communication Diagrams illustrate how different components of the University Portal communicate with each other through numbered messages.

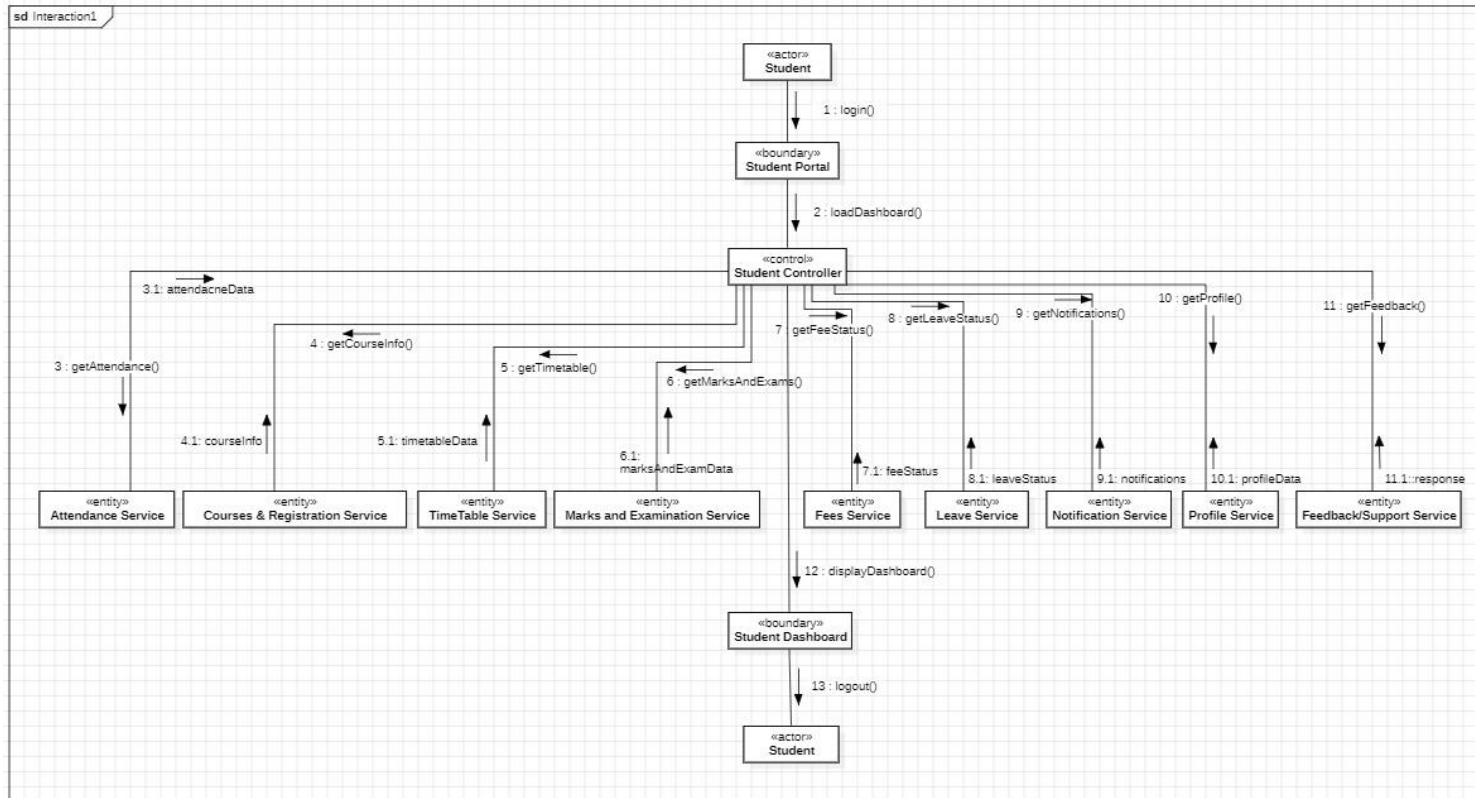
Overall University Portal Communication Diagram

Shows the authentication process, user verification, role identification, and loading of the appropriate Student, Faculty, or Administrator portal.



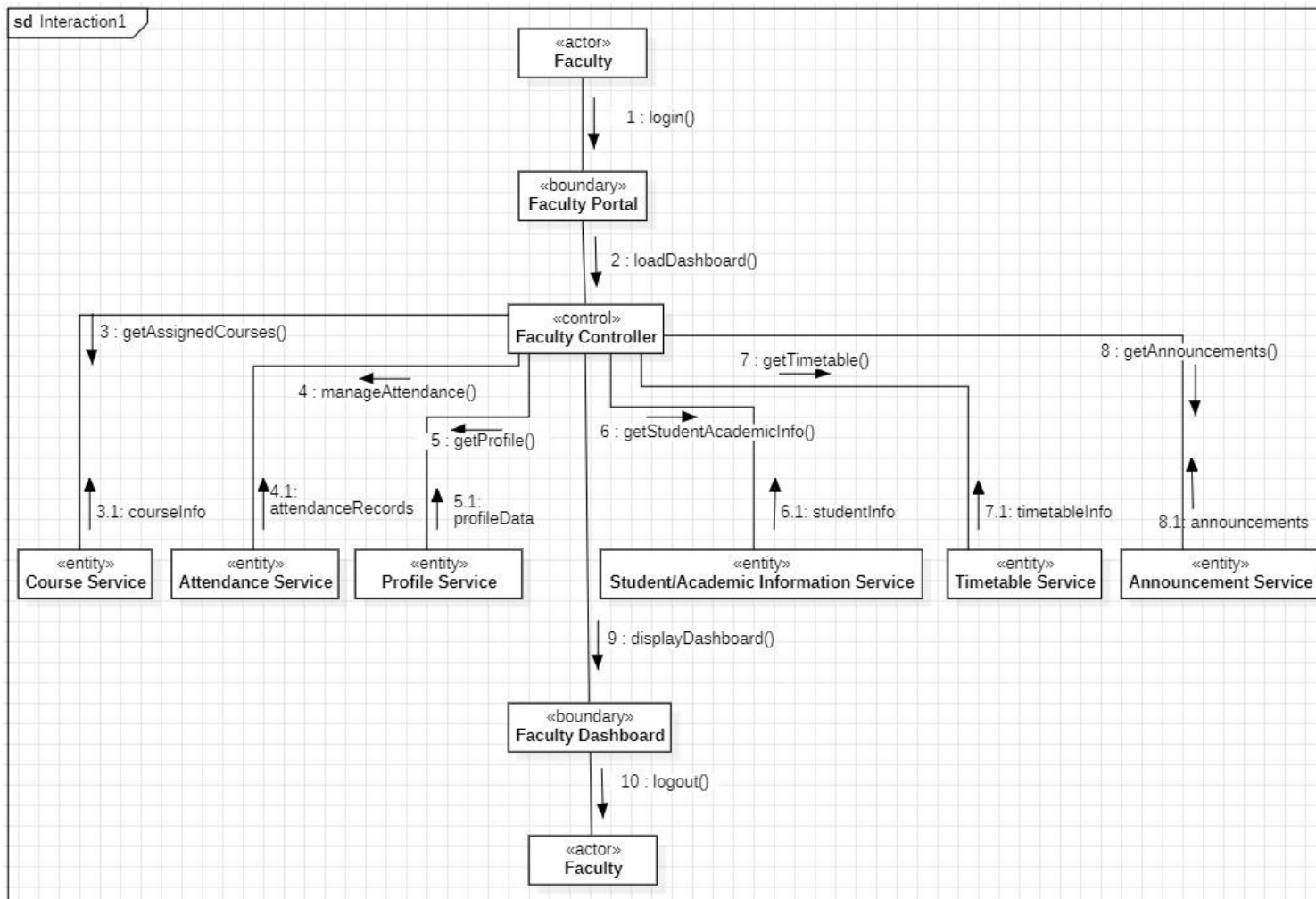
Student Module Communication Diagram

Shows communication between the Student Controller and services for courses, attendance, timetable, marks and examinations, fees, leave, notifications, profile, and feedback/support.



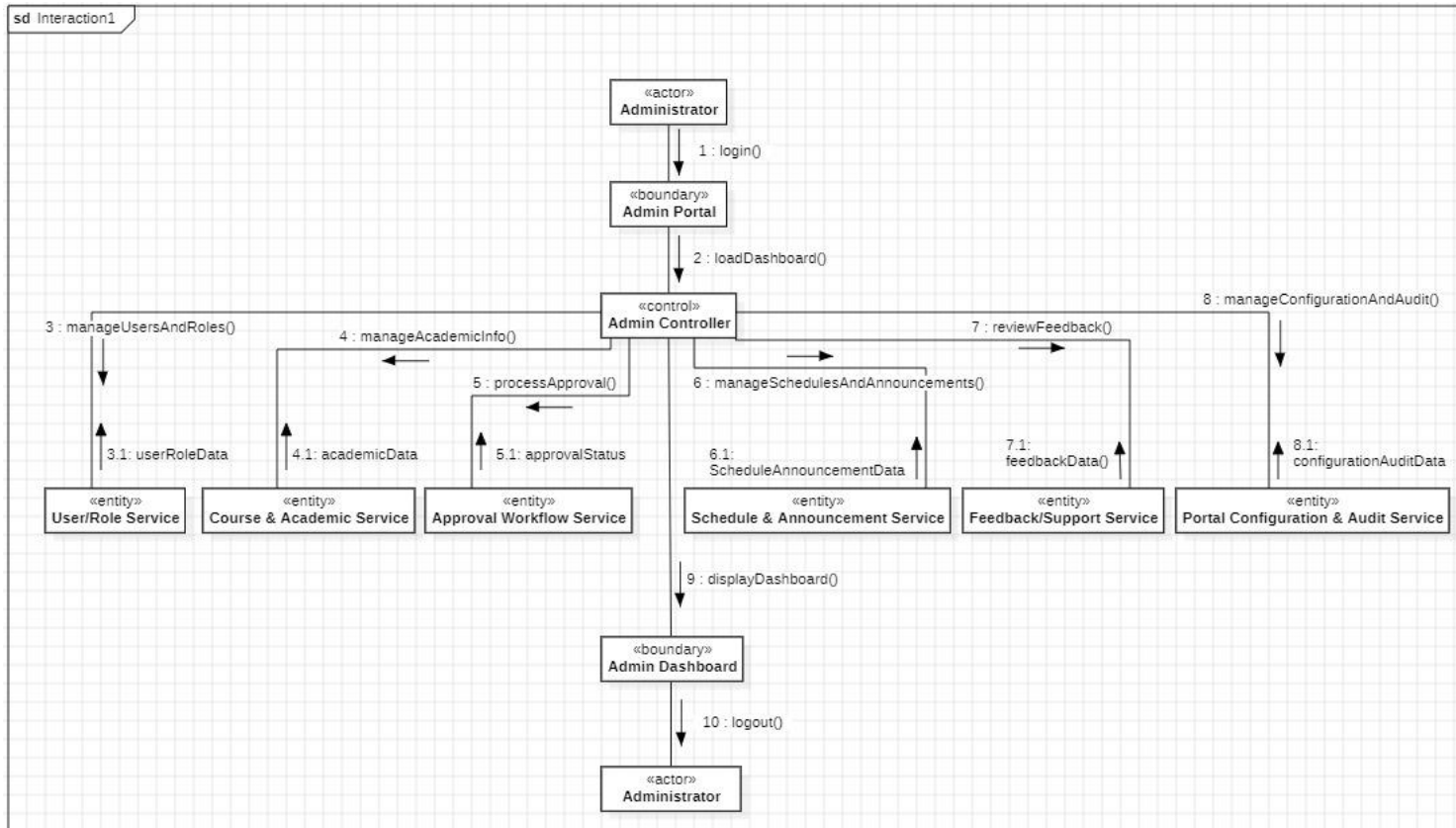
Faculty Module Communication Diagram

Shows communication for assigned courses, attendance management, profile, student/academic information, timetable, and announcements.



Administrator Module Communication Diagram

Shows communication for user and role management, academic information, approval workflows, schedules and announcements, feedback/support, and portal configuration and audit.



Overall Message Flow Explanation

1. OVERALL MESSAGE FLOW

The communication flow of the University Portal begins when a user—Student, Faculty, or Administrator—attempts to log into the system. The login request is passed to the Login Page and then to the Authentication Controller for verification.

Message	Flow	Explanation
1. login()	User → Login Page	The Student, Faculty, or Administrator enters their login credentials.
2. authenticateUser()	Login Page → Authentication Controller	The login request is sent to the Authentication Controller for authentication.
3. verifyUser()	Authentication Controller → User Database	The system verifies the user's credentials against the stored user information.
4. getRole()	Authentication Controller → Role Manager	The system identifies the role assigned to the authenticated user.
5. returnUserInfo	User Database → Authentication Controller	The user information is returned after successful verification.
6. returnUserRole	Role Manager → Authentication Controller	The user's role—Student, Faculty, or Administrator—is returned.
7. [role]: loadPortal()	Authentication Controller → Appropriate Portal	Based on the identified role, the system loads the corresponding Student, Faculty, or Administrator Portal.

3. PORTAL RESPONSIBILITIES AFTER AUTHENTICATION

STUDENT PORTAL

- Courses and course registration
- Attendance
- Timetable
- Marks and examinations
- Fees
- Leave / outing
- Notifications
- Profile
- Feedback / Support

FACULTY PORTAL

- Assigned courses
- Attendance management
- Student / academic information
- Timetable
- Announcements
- Profile

ADMINISTRATOR PORTAL

- User and role management
- Course and academic information
- Schedule and announcements
- Approval workflows
- Feedback / support
- Portal configuration and audit

4. CONCLUSION

The overall communication flow ensures that only authenticated users can access the portal, and the role returned by the authentication system determines which module is loaded. This provides the foundation for the Student, Faculty, and Administrator interactions.

2. OVERALL COMMUNICATION FLOW DIAGRAM (REFERENCE)

